# HealthEx Scaling Design Exercise

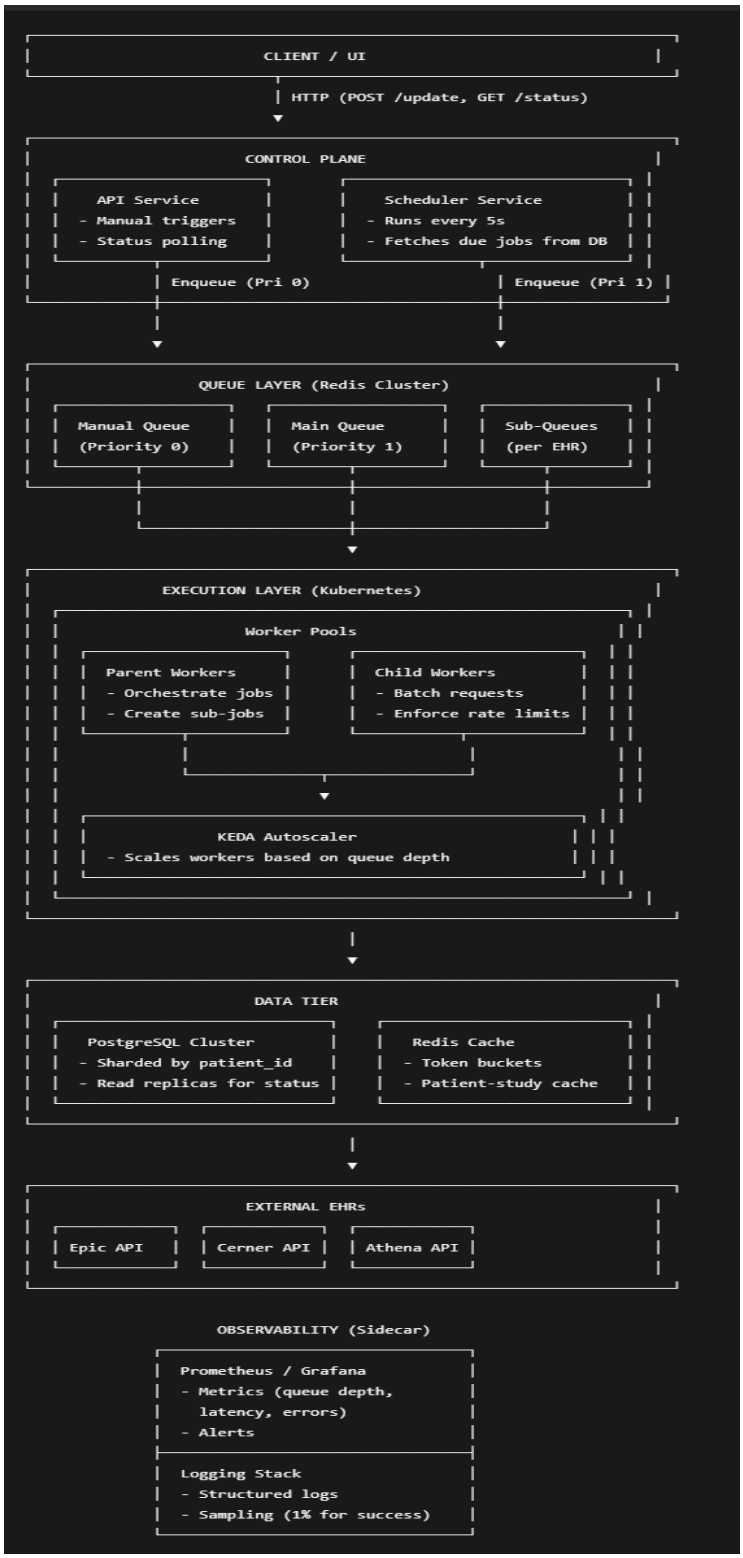
Nishchal Agrawal

codebase: https://github.com/nishchalAgr/healthex_orchestrator_exercise

## 1. Current State & Bottlenecks

| **Component** | **Current Limitation** |
|---|---|
| Database | SQLite (single file, write locks, no horizontal scaling). |
| Scheduling | Scans patient_studies table on startup; rescheduling done by workers. |
| Queue | Single Redis instance (BullMQ), limited by memory and network I/O. |
| Rate Limiting | Not implemented (relies on mock delays). |
| Workers | Fixed count (N=5–10) running in the same process as the API. |

## Component Design



## 2. Data Tier / State Management

While SQLite was chosen for its simplicity, it is an embedded, serverless database that will run into issues as we scale. Rather, we can use a more robust client-server DB engine like PostgreSQL. This would also unlock certain scaling strategies:

1. shard by PatientId so that database operations can be distributed across multiple nodes.
2. read replicas to better handle a large volume of GET /status calls

We can also cache frequently read patient-study rows. Cached rows would be invalidated on manual and scheduled refreshes.

## 3. Job Queue

**Distributed Queue**

A single redis instance likely cannot handle millions of queued jobs. To address this, we can use something like *redis cluster* to automatically distribute jobs across multiple nodes. One downside of this approach is that *redis cluster*, in the interest of performance, does not guarantee strong consistency. In other words, there is a chance that a job could be lost but reported as successfully added to the queue. This issue will be handled in the next section (scheduling).

**Manual refresh request prioritization**

The current design does prioritize manual refresh requests (/update endpoint) by ensuring that manually requested jobs are picked up before any other job. However, if an update request is made while all workers are busy, it will still have to wait for a worker to free up. To address this, we can have a separate queue with dedicated workers just for manually requested jobs.

## 4. Scheduling

Currently, we rely on workers to schedule the next job after finishing executing the current one. An issue with this approach is that if a worker crashes while executing a job, not only will the current refresh fail, but all future refreshes for that particular patient-study will not be scheduled. This strategy also fails to account for situations in which the worker fails to schedule the next job (see above section).

Rather we should have a dedicated scheduler service that runs periodically (ex. every five seconds)

1. Add a `next_run_at` column to patient-study table that the worker is responsible for updating
2. Scheduler fetches all rows where `next_run_at < Date.now()` (limit 1000)
3. Scheduler adds a job for each row

This way, if a worker fails to refresh a row for any reason, it will be picked up immediately by the scheduler because `next_run_at` will be out of date.

## 5. Workers (refresh execution)

Workers are only spun up on server startup, meaning the total number of workers is static regardless of how many jobs are waiting in queue.

**Horizontal Autoscaling**
We can use an autoscaler like Kubernetes Event-Driven Autoscaler (KEDA) to dynamically create workers based on the queue depth.

- Also creates new processes for each worker rather than having all of them on the server process

**Batch Fetching**
If available in the EHR API, we should bundle requests together in order to reduce the total number of API calls made, which in turn reduces the number of workers needed. To achieve this, we could modify the design like so:

1. Two types of job queues
    a. The main queue - handles refresh jobs
    b. A “sub” queue for each EHR endpoint - handles data requests for a given EHR
2. Workers pull parent jobs from the main queue (1.a)
    a. for each data source we need to query, add a “child” job to the corresponding queue (1.b)
    b. all child jobs must be fulfilled before parent job can be completed
3. Sub queues have their own dedicated workers
    a. Each worker takes the top X child jobs in the queue, compiles it into one batch request, makes a single EHR api call, updates the DB, and marks the child jobs as complete.

## 6. Rate limiting

(also in github readme)

Currently, the orchestrator does not explicitly take EHR rate limits into account. To solve this issue, we could modify the current design in such a way:

1. Using redis, create a token queue per EHR endpoint
2. Refill each queue at the rate limit for the given EHR endpoint. For example, if Athena has a rate limit of 30RPM, then the Athena queue can have at most 30 tokens and 1 token would be added every 2 seconds
3. Before making a request to an endpoint, the Worker will repeatedly try to remove a token from the queue until one is available.

## 7. Observability

**Metrics Monitoring** (Prometheus + Grafana)

1. Queue depth/size
2. Number for workers and worker utilization (% that are busy)
3. Waterfall latency - capture latency at every stage (p50, p95, p99)

    a. ie. total worker latency, api call latency
4. Number of errors and exceptions

Setup up alerts to notify if metrics behave in concerning ways

1. ie. sustained spike in errors or latency

Furthermore, every step of job processing should be logged so that we can have a “stacktrace” to help with investigating job failures. While this will result in a large amount of data, we can:

1. use a short retention period, as we would expect critical failures to be investigated as they happen
2. aggressively sample non-failing jobs (ie. 1%)